Codex (GPT-5.4) as an Agentic AI Coding Assistant: A Practitioner Benchmark Study in Production Software Development

Manee & Sushma
ACME Development Team

***Abstract*—Codex (OpenAI, 2026), powered by the GPT-5.4 architecture, is OpenAI's agentic coding assistant. It runs tasks in cloud sandboxes, uses git worktrees to keep work isolated, compacts its context natively to support long sessions, and includes a two-layer security model. We evaluate it as an AI coding assistant through a practitioner benchmark involving real feature implementation in a production SaaS codebase, across eight areas: workflow integration, code generation quality, contextual reasoning, architectural adherence, problem-solving capability, documentation generation, operational characteristics, and overall productivity impact.**

Using a 26-criterion rubric (weighted mean: 4.50/5.00, 90.0%), benchmark data verified against primary sources, and developer community feedback, we find that Codex is strong at analytical problem solving, fast execution on well-defined tasks, long-running autonomous sessions, and technical debt clean-up. Its SWE-Bench Pro score (57.7% vs. Claude Opus 4.6's 53.4%) shows it handles novel engineering problems better, and its OSWorld-Verified score (75.0%) puts it ahead of the human expert baseline (72.4%) on desktop automation. Key weaknesses include a tendency to push ahead destructively on vague tasks, IP concerns from cloud-hosted code execution, and rate-limit friction on the Pro subscription tier. These findings give development teams a practical basis for evaluating Codex as part of their AI toolchain.

***Index Terms*—Codex, GPT-5.4, agentic AI, coding assistant, benchmark, git worktrees, cloud execution, context compaction, SWE-Bench Pro**

I. Introduction

AI-assisted software development has changed significantly in 2026. The industry has shifted away from single-context autocomplete tools toward autonomous systems that can refactor large codebases, run terminal commands, and navigate desktop environments with minimal human input. A key part of this shift is OpenAI's GPT-5.4 architecture, which merged the older GPT-5.3-Codex model into a single system and launched a redesigned Codex application alongside it.

Codex stands out from competing tools in a few concrete ways: it runs tasks in cloud containers rather than on the developer's machine, uses git worktrees to keep each task isolated so multiple projects can run at the same time, and includes a compaction capability that lets it work through sessions much longer than its context window would normally allow.

A. Codex: Technical Overview

Codex can be accessed through multiple channels:

- **Standalone application**: A desktop app that replaced the older CLI tool. It supports custom skills, automation pipelines, and native Machine Context Protocol (MCP) server support [18]
- **VS Code extension**: Works with VS Code forks including Cursor and Windsurf [18]
- **JetBrains IDE integration**: Supports IntelliJ, PyCharm, Rider, and WebStorm [18]
- **CLI (open-source)**: Available on GitHub, configured through layered `.codex/config.toml` files

The model's most notable characteristics include:

- **Cloud worktree execution**: Tasks run in isolated OpenAI-managed containers, with a separate git worktree per task. Multiple projects can run in parallel without using local machine resources [19]
- **Native context compaction**: GPT-5.4 is trained to work across multiple context windows by compacting earlier parts of a session. This is not simple summarization; it is a core model capability that allows sessions to run for 24 hours or more while using roughly 30% fewer reasoning tokens than earlier models [20]
- **1.05M token context window (experimental)**: Codex includes experimental support for a 1.05M token context window, configurable via `model_context_window` and `model_auto_compact_token_limit`. The standard production window is 272K tokens; requests that exceed it are charged at $2\times$ the normal rate. This is combined with a Tool Search feature that loads tool definitions on demand rather than upfront, reducing input token use by roughly 47% for agentic tasks [20]
- **SWE-Bench Pro score**: 57.7% on novel, non-gameable engineering tasks, ahead of Claude Opus 4.6 (53.4%) by 4.3 percentage points [8]
- **Two-layer security model**: Sandbox Mode controls what the agent can technically do, and the Approval Policy Engine decides when it must ask for permission before acting. Both work independently and use OS-level en-

forcement on macOS (Seatbelt), Linux (`bubblewrap`), and Windows (WSL2) [18]
- **Fast Mode**: Delivers up to 1.5× faster output using the same underlying model, without a quality trade-off. Useful for keeping developer flow on straightforward tasks [13]
- **AGENTS.md project instructions**: A file-based instruction system that GPT-5.4 is fine-tuned to follow. Instructions are version-controlled and more reliably followed than free-form session prompts

B. Research Questions

This paper addresses three primary research questions:

Q1: How effectively does Codex (GPT-5.4) work as an AI coding assistant in a structured, convention-driven production codebase, in terms of analytical problem solving, sustained execution, and architectural reasoning?

Q2: What are the observable operational characteristics of Codex, including its cloud execution model, context compaction behavior, rate-limiting, and cost, that affect its suitability for daily professional use?

Q3: How do Codex's key features, specifically git worktrees, cloud sandboxing, native compaction, and Tool Search, give it an edge over single-context and locally-executed tools?

II. Background and Related Work

A. Benchmarks for AI Code Generation

Evaluation of AI coding tools has gone through several rounds of improvement. HumanEval [1] was an early standard using 164 Python problems, but has been criticized for being too narrow and prone to data contamination [3]. SWE-bench [2] improved on this by using real GitHub issues that require changes across multiple files.

The SWE-Bench ecosystem has since split into two variants. SWE-Bench Verified tests well-documented, standardized pull request resolution. Here, GPT-5.4 (∼80.0%) and Claude Opus 4.6 (81.4%) are roughly tied, with Opus 4.6 slightly ahead on consistent execution of known patterns [11]. The more meaningful metric for real-world use is **SWE-Bench Pro**, which is designed to prevent pattern matching by requiring agents to reason through genuinely novel problems. On this benchmark, GPT-5.4 scores 57.7% against Claude Opus 4.6's 53.4%, a 4.3 percentage point gap that reflects better first-principles reasoning on unfamiliar problems [8].

OSWorld-Verified tests autonomous desktop operation, including interacting with GUIs and completing multi-step workflows. GPT-5.4 scores 75.0%, ahead of both the human expert baseline (72.4%) and Claude Opus 4.6 (72.7%), supported by a vision system that handles images with over 10 million pixels without compression loss [9].

B. Developer Productivity Studies

A large randomized controlled trial with over 4,000 developers found that GitHub Copilot cut average task completion time by roughly 55% on new coding work, with junior developers seeing the biggest gains [4]. McKinsey Global Institute found a 26% increase in developer output when using AI tools, measured by story points per sprint [5].

A contrasting result from METR [6] found a 19% *increase* in task completion time for experienced developers working on complex, unfamiliar codebases. This points to the fact that AI coding tools do not help equally across all situations; gains depend heavily on how well the task is defined and whether the developer stays in control of the architecture. This finding also reinforces the value of a well-written `AGENTS.md`: clear constraints reduce ambiguity, which is the main driver of both slower workflows and Codex's known tendency to charge ahead on underspecified tasks.

C. Comparative Landscape

As of 2026, agentic coding tools split along two main lines:

- **IDE-first vs. cloud-first**: Cursor and Copilot integrate directly into the editor and work in a single context. Claude Code and Codex work at the repository level, with Codex uniquely offering cloud container execution
- **Parallel orchestration vs. sustained sequential execution**: Claude Code can spin up 5 to 6 specialized agents working in parallel, with automatic QA checks when changes are made. Codex focuses on sustained single-agent execution and delegates simpler subtasks to cheaper models

Codex is the only current tool that runs tasks in cloud containers with git worktree isolation per task, giving each piece of work its own clean copy of the repository. Claude Code runs locally and manages parallelism through sub-agents rather than container isolation.

III. Methodology

A. Evaluation Design

This study uses a practitioner observational design [7]: developers used Codex as the primary AI assistant over a structured evaluation period on real production tasks. The outputs and experience were assessed using three independent instruments, plus external benchmark data and developer community feedback from forums, blogs, and comparison articles.

B. Evaluation Environment

The evaluation was conducted on a production SaaS platform for social media content scheduling. The codebase is a monorepo comprising:

- **Backend**: Python FastAPI following Clean Architecture (API → Controller → UseCase → Domain → Infrastructure)
- **Frontend**: Flutter mobile application with Clean Framework and Riverpod v3 state management

The codebase is a realistic test environment: it has non-trivial architectural conventions, strict layer boundaries, an established set of patterns Codex needed to find and follow, and a running Docker Compose stack for integration testing. An `AGENTS.md` file at the repository root provided explicit architectural rules, key file paths, build and test commands, and a list of negative constraints.

C. Evaluation Tasks

Codex was evaluated across multiple task categories over a structured evaluation period:

Phase 1: Setup and initial configuration, Fast Mode vs. Standard Mode comparison, cloud worktree initialization and parallel task execution

Phase 2: Context retention and `AGENTS.md` constraint fidelity across multi-hour sessions, including monitoring behavior through compaction events

Phase 3: Full-stack feature implementation (Pinterest default board selection, 24 files modified or created) run through cloud sandbox with git worktree isolation

Phase 4: Sustained execution sprint, including technical debt identification and automated legacy refactoring over multi-hour autonomous sessions

Phase 5: Bug identification in existing codebase, cross-model audit workflow (Codex reviewing Claude-generated code), and documentation generation

D. Evaluation Instruments

Instrument 1: Quantitative Rubric. Developers rated Codex on a 26-criterion rubric across eight categories: Workflow and Setup, Code Generation Quality, Context and Memory Management, Architecture and Planning, Problem Solving, Operational Characteristics, Documentation, and Overall Productivity Impact. Each criterion was scored 1 (critically deficient) to 5 (excellent).

Instrument 2: AGENTS.md Constraint Compliance Evaluation. A structured test of how well Codex followed project-level architectural constraints from `AGENTS.md`. The test measured adherence to Clean Architecture layer boundaries, naming convention consistency, build command compliance, and whether constraints held up after compaction events. This instrument is specific to Codex's instruction system and does not have a direct equivalent for tools without comparable file-based project instructions.

Instrument 3: External Benchmark Triangulation. Observed behavior was compared against published benchmarks (SWE-Bench Pro, OSWorld-Verified, Terminal-Bench 2.0, Toolathlon, BrowseComp, MMMU Pro), developer community feedback, and comparative tool reviews. All benchmark figures were checked against primary sources including BenchLM [10], Vellum AI [11], official OpenAI and Anthropic documentation, and the MorphLLM SWE-Bench Pro leaderboard [8].

Measurement note. Rubric scores reflect developer observations from this evaluation and should be read as internal reference points. Benchmark figures from third-party sources depend on their specific evaluation setup and may differ from other published results.

IV. Results

A. Quantitative Rubric Scores

Table I presents category-level scores; the full 26-criterion breakdown is in Table II. The weighted mean was 4.50 / 5.00 (90.0%).

TABLE I
CATEGORY-LEVEL RUBRIC SCORES (1 TO 5 SCALE)

Category	Mean
Workflow and Setup	5.00
Code Generation Quality	4.50
Context and Memory	4.50
Architecture and Planning	4.75
Problem Solving	5.00
Documentation	4.50
Operational / Non-Functional	3.60
Overall Productivity Impact	5.00
Overall Weighted Mean	**4.50**

TABLE II
FULL 26-CRITERION RUBRIC SCORES

Category	Criterion	Score
Workflow & Setup	Setup & Configuration	5
	Learning Curve	5
Code Generation	Accuracy & Correctness	5
	Feature Creation	5
	Maintainability	4
	Code Consistency	4
Context & Memory	Instruction Following	4
	Context Awareness	5
	Context Window Limits	4.5
	Token Efficiency	4
	Project-Level Instructions	5
Arch. & Planning	Feature Planning	4
	Architecture Summary	5
	Multi-Step / Role-Based	5
	Sub-Agent Support	5
Problem Solving	Bug Finding & Fixing	5
	Iterative Refinement	5
	Handling AI Mistakes	5
	Tech Debt Identification	5
Operational	Security & IP	1
	Cost Considerations	4
	Response Time / Latency	3
	IDE Integration	5
	Rate Limiting	5
Documentation	Documentation & Comments	4.5
Overall Impact	Productivity Observations	5

B. External Benchmark Triangulation

Table III shows Codex's scores across major benchmarks, compared against Claude Opus 4.6. All figures have been checked against primary sources.

C. AGENTS.md Constraint Compliance Evaluation

Codex followed the project-level constraints in `AGENTS.md` consistently throughout the evaluation. The instruction hierarchy, where CLI flags override project files, which override global config, which overrides system defaults, held up across all sessions including ones where earlier context had been compacted away.

Key findings from the compliance evaluation:

TABLE III
CODEX (GPT-5.4) VS. CLAUDE OPUS 4.6: KEY BENCHMARK RESULTS

Benchmark	Codex (GPT-5.4)	Claude Opus 4.6	Domain
SWE-Bench Verified	∼80.0%	81.4%	Standard issue resolution
SWE-Bench Pro	**57.7%**	53.4%	Novel engineering logic
OSWorld-Verified	**75.0%**	72.7%	Desktop automation (Human baseline: 72.4%)
Terminal-Bench 2.0	**75.1%**	65.4%	CLI & agentic terminal operations
Toolathlon	**54.6%**	47.2%	External tool invocation
BrowseComp	82.7%	**84.0%**	Multi-step agentic web search
MMMU Pro (w/ tools)	**81.2%**	77.3%	Visual & multimodal reasoning
MRCR v2 (1M context)	∼70.0%†	**76.0%**	Long-context retrieval

†GPT-5.4 1M-context MRCR score unverifiable from indexed sources. Note: 1M context is experimental in Codex; standard window is 272K. Requests exceeding 272K are billed at 2× the normal rate.

- Clean Architecture layer boundaries were maintained across the 24-file feature implementation without any per-prompt reminders
- Naming conventions and state management patterns set in `AGENTS.md` survived compaction events. File-based instructions proved much more durable than free-form session instructions under the same conditions
- Negative constraints such as "do not modify files outside this directory" were respected in both Worktree and Cloud modes, with the git worktree boundary acting as an additional technical safeguard
- Feature Planning scored 4/5 rather than 5/5, because Codex occasionally skipped the planning step on ambiguous sub-tasks and went straight to writing code, reflecting the rushing tendency noted in practitioner reviews [14]

The `AGENTS.md` hierarchy can be configured at multiple levels: global organizational standards at the repository root, with more specific overrides deeper in the directory tree for individual microservices. This approach was consistently rated by evaluators as a clear operational advantage over tools that require re-stating constraints in every session.

D. Observed Workflow Behavior

1) Cloud Worktree Execution and Parallel Isolation: Codex supports three execution modes: Local (runs on the developer's machine), Worktree (creates an isolated git worktree locally), and Cloud (runs in OpenAI-managed containers). The Cloud mode uses a two-phase design with a clear security boundary:

- **Setup Phase**: Internet access is enabled for installing dependencies; all configured secrets are available
- **Agent Phase**: Internet is off by default; secrets are removed before this phase starts; all network traffic goes through a proxy

Container state is cached for up to 12 hours and cleared automatically when setup scripts, environment variables, or secrets change. This gives repeatable environments without extra setup cost per task. Subagents also run in their own isolated containers with limited file access and no network, keeping parallel subtasks properly separated.

In the parallel isolation test, five concurrent tasks ran across two repositories. Each task worked on its own git worktree with no conflicts, and results were available for review as each task finished. This "delegate and review" workflow is not possible with single-context tools like Copilot or Cursor.

2) Context Compaction and Long-Horizon Execution: GPT-5.4's context compaction is built into the model itself rather than applied on top of it. When a session gets close to the context limit, the model prunes earlier history while keeping the parts it still needs, such as architectural decisions and key constraints, so work can continue without losing track of the goal [20].

In practice, this results in:

- A 20–40% reduction in total token use for long sessions over 100,000 tokens
- About 30% fewer reasoning tokens for the same quality of output compared to earlier models
- OpenAI has documented internal runs exceeding 24 hours of continuous autonomous execution

The Token Efficiency criterion scored 4/5. The compaction capability is genuinely useful, but some developers have reported that mid-task compaction can cause the agent to lose nuanced context it had not yet written down, effectively starting that part of the task over [18].

The **Tool Search** feature addresses a separate token problem. Instead of including all tool definitions in every API request, GPT-5.4 loads them on demand during a session. This cuts input token use by roughly 47% on agentic tasks, which meaningfully lowers the running cost of long automated pipelines [20].

3) Fast Mode Velocity: Codex's Fast Mode produces output up to 1.5× faster using the same model, with no quality reduction. In developer reviews, this was consistently described as the most useful improvement in the GPT-5.4 release: "I would usually rather finish the task and spend the quota than save quota while an agent crawls through the job" [13].

Fast Mode works best on tasks with clear, well-defined requirements. The Response Time / Latency rubric score of 3/5 reflects the flip side: on complex multi-file tasks that need more careful reasoning, Codex does not spend the same amount of upfront planning time that Claude Code does, which can mean more correction needed later. The practical approach is to use Fast Mode for execution work on well-specified tasks and standard mode for planning phases.

4) Analytical Problem Solving and Bug Identification: A consistent pattern in practitioner reviews is that Codex is exceptionally thorough when reviewing code or debugging. In direct comparisons on the same codebase, developers report that Claude Code typically finds one to two high-level architectural issues, while GPT-5.4 finds four to five including subtle edge cases, type mismatches, nullability issues, and state inconsistencies that even experienced reviewers miss [12].

This has been demonstrated in real production contexts: Codex has been used to track down complex WebGL rendering bugs and deep Windows memory crashes, with a documented success rate above Claude Opus 4.6 on the same tasks [12]. During the evaluation, Bug Finding, Iterative Refinement, Handling AI Mistakes, and Tech Debt Identification all scored 5/5.

The long-running execution capability adds to this. With a well-written `PLANS.md` file, Codex has been documented running continuously for over 46 minutes, reading files, committing changes, running tests, fixing its own errors, and pushing code without losing context [12]. For legacy code clean-up, the model has refactored over 12,000 lines of undocumented Python in a single session, fixing race conditions that had been in production for months.

5) Destructive Execution Risk: The most significant safety concern from practitioner feedback is what reviewers call *destructive execution*: when given vague or contradictory instructions, Codex tends to push forward and make assumptions rather than stopping to ask for clarification [14]. In documented cases, it has completed a portion of a refactoring correctly and then overwritten working code with incorrect output, leaving zero net progress and requiring a full rollback [15].

This contrasts with Claude Code's behavior of pausing on ambiguous instructions to discuss intent before acting. The root cause is not a capability gap; Codex understands the code it is working with. The problem is that it does not automatically check whether its plan is complete before executing. The recommended mitigations are:

- Writing comprehensive `AGENTS.md` files that include explicit "do not" rules and permission boundaries
- Setting sandbox mode to `workspace-write` rather than `danger-full-access` for sensitive sessions
- Using the `on-request` approval policy for any session involving irreversible changes
- Running in Worktree or Cloud mode: if destructive changes happen inside a worktree, the main branch is untouched and the worktree can simply be deleted

The git worktree design is a particularly practical safeguard here. It turns a potentially serious mistake into a discardable branch, rather than damage to the working tree.

6) Rate Limiting Behavior: Where Claude Code scored 1/5 on Rate Limiting in the same evaluation series, Codex scored 5/5. On standard Plus tier plans, rate limiting was not a problem during the evaluation period.

The Pro tier is a different story. It provides roughly 40 minutes of reasoning capacity per 5-hour window. A single complex task lasting 20 minutes can use half the available allowance, which makes sustained professional use on the Pro tier difficult [16]. There is also no mid-tier option between the $20 Plus and $200 Pro plans, unlike Anthropic which offers a $100 Max tier, leaving Pro users with limited choices when their quota runs out.

E. Cross-Model Audit Workflow

A practical workflow that has emerged in the developer community is using Codex to audit Claude-generated code, and Claude to audit Codex-generated code [12]. GPT-5.4's detailed analytical review catches Claude's tendency to over-engineer solutions, often suggesting simpler and more secure alternatives. Claude Opus 4.6 in turn catches architectural gaps and missing error handling in Codex's output.

OpenAI has supported this officially by releasing a Codex plugin for Claude Code, allowing cross-provider review in a single session [17]. Running the two models in alternating review cycles, where each reviews the other's output, consistently produces better results than either model working alone [12].

F. Documentation Generation

Documentation generation scored 4.5/5. Codex reliably produces clear, accurate documentation including inline comments, API doc comments, and README files. The Architecture Summary criterion (5/5) reflects its ability to analyze complex systems and produce structured summaries, including hardware-oriented designs like Post Fusion inference systems with quantized sub-models on embedded hardware.

V. Discussion

A. The Analytical Depth Advantage

The clearest finding from both the rubric and benchmark data is that Codex is better at solving novel problems. The 4.3 percentage point lead on SWE-Bench Pro (57.7% vs. 53.4%) is meaningful because this benchmark is specifically designed to prevent models from relying on memorized answers. It measures genuine reasoning, not pattern recall. In practice, this means Codex finds more bugs per review, surfaces a wider range of technical debt, and produces solutions that do not depend on having seen similar problems before [12].

For teams dealing with large, poorly documented legacy codebases, especially those with complex concurrency logic, OS-level integrations, or tangled backend APIs, this makes Codex the stronger choice for debugging and execution work.

B. Cloud Architecture as a Competitive Differentiator

Codex's cloud execution model, built on git worktree isolation and containerized tasks, enables workflows that locally-executed tools simply cannot match. Three advantages stand out:

Parallel project execution: Multiple tasks across different repositories can run at the same time, each in its own container, without competing for local resources. A developer can kick

off parallel refactoring tasks across multiple microservices and check in on results as they finish.

Repeatable environments: Container state is built deterministically from a setup script and cached. This gives consistent, reproducible runs that local tools relying on the developer's filesystem cannot provide. This matters in regulated environments where audit trails and repeatable builds are required.

Clear security boundary: The two-phase execution model (internet-enabled setup, offline agent phase) means code is not being sent out during execution. This addresses part of the IP concern behind the Security & IP score of 1/5, which mainly reflects anxiety about code leaving the organization's infrastructure at all.

C. Native Compaction vs. Session Management Workarounds

Other tools handle long sessions through workarounds: Claude Code isolates context using sub-agents, and developers manually summarize progress at key points. Codex handles this differently by letting the model itself manage what it keeps and what it drops. Developers do not need to architect sessions around context limits.

The 47% token saving from Tool Search adds to this, making Codex the more cost-efficient option for high-volume automated pipelines where token consumption drives cost.

The main caveat is that compaction can occasionally drop context the model had not yet recorded anywhere. Teams running complex multi-file refactoring sessions should use `PLANS.md` files to write down task state externally, so key decisions survive a compaction event.

D. The Destructive Execution Problem and Its Mitigations

The destructive execution pattern is a genuine, documented issue [14], [15]. To be clear about what it is: Codex understands the code it works with. The problem is that it does not automatically verify its plan is complete before starting. When instructions are vague or conflicting, it charges ahead rather than asking questions.

The fix is straightforward but requires upfront work. A combination of thorough `AGENTS.md` constraints, `workspace-write` sandbox mode, `on-request` approval policy, and git worktree isolation creates a safety net that keeps the impact of any single bad execution to a discardable branch. Teams that set this up properly report a much better experience than those running Codex in default configuration on loosely defined tasks.

E. Economic Considerations and Total Cost of Ownership

Codex Standard is significantly cheaper than Claude Opus 4.6 on raw API pricing: 50% less on input tokens ($2.50 vs. $5.00 per million) and 40% less on output tokens ($15.00 vs. $25.00 per million). Cached input is also cheaper at $0.25/M compared to $0.50/M for Opus 4.6, which helps on iterative work over large codebases. Table IV shows the full comparison.

TABLE IV
API PRICING COMPARISON (PER 1M TOKENS, VERIFIED APRIL 2026)

Model	Input	Output	Cached	Context
Codex (GPT-5.4)	$2.50	$15.00	$0.25	272K (1.05M exp.)
Claude Opus 4.6	$5.00	$25.00	$0.50	1.00M
Codex Pro	$30.00	$180.00	N/A	>272K

Combined with Tool Search's 47% token reduction, Codex Standard is the most cost-effective choice for high-volume automated testing, prototyping, and bulk refactoring work with well-written `AGENTS.md` guardrails.

That said, total cost shifts on complex work. Codex's slightly lower accuracy on coordinated multi-file tasks leads to more retries, more rollbacks, and more developer time spent fixing incorrect output. When senior developer time is factored in, Claude Opus 4.6 can end up cheaper for work where getting the architecture right the first time matters most.

F. The Dual-Wielding Recommendation

The most consistently supported finding from developer community analysis is that using both tools together produces the best results [12]. Claude Opus 4.6 and Codex have complementary strengths:

- **Claude Opus 4.6 for architecture**: UI generation, large multi-file refactoring, handling ambiguous requirements, and running automatic QA checks through its Agent Teams feature
- **Codex for execution and review**: Backend logic, concurrency debugging, technical debt, reviewing Claude-generated code, and running high-volume automated pipelines

The best teams route tasks based on what each model is good at: depth and design to Claude, speed and analytical verification to Codex. OpenAI's official Codex plugin for Claude Code supports this directly by allowing cross-provider review in a single integrated session [17].

G. Terminal Regression: A Practical Note

For teams that rely heavily on terminal operations, GPT-5.4 scores 75.1% on Terminal-Bench 2.0, which is actually slightly behind its predecessor GPT-5.3-Codex at 77.3% [9]. Merging the Codex model into GPT-5.4 brought big gains in visual reasoning and general capability but came with a small regression in raw terminal performance. Teams whose workflows depend heavily on SSH, complex Git scripting, shell debugging, or build system automation should be aware that GPT-5.3-Codex is still the better choice for purely terminal-heavy use. For mixed workflows, the broader gains from GPT-5.4 outweigh this regression.

VI. LIMITATIONS

Single project, limited task scope. All conclusions come from one codebase. How well they generalize to other languages, frameworks, or team sizes is unknown.

No controlled comparison. Without running the same tasks on competing tools under the same conditions, productivity claims cannot be directly attributed to Codex.

Benchmark sourcing. Some benchmarks, particularly SWE-Bench Pro and GDPval, are taken from secondary aggregator sources where the exact evaluation setup may differ from primary published conditions. GDPval scores are shown as percentages in secondary sources, but the primary scoring system is Elo-based (GPT-5.4: 1672; Claude Opus 4.6: 1606); the percentage figures are approximations. The MRCR v2 score for GPT-5.4 at 1M context could not be confirmed from any indexed source at time of writing.

Snapshot validity. Codex is actively developed. Rate limits, pricing, compaction behavior, and benchmark scores may all change with future updates.

Destructive execution variability. How often destructive execution occurs depends heavily on how well the task is specified and how the sandbox is configured. Evaluations run with tight `AGENTS.md` constraints will see far fewer incidents than those run with vague or minimal instructions.

VII. Conclusion

This paper evaluated Codex (GPT-5.4) as an AI coding assistant through a practitioner benchmark grounded in production feature implementation. We address our three research questions as follows:

Q1 (Core Capability). Codex works well as an AI coding assistant in structured, convention-driven codebases, particularly for analytical problem solving, technical debt identification, and sustained autonomous execution. Its SWE-Bench Pro score of 57.7% confirms a genuine advantage on novel engineering problems. The 24-file feature implementation completed with full Clean Architecture integrity, and Bug Finding, Iterative Refinement, Handling AI Mistakes, and Tech Debt Identification all scored 5/5. The main caveat is the destructive execution tendency on vague tasks, which requires investment in `AGENTS.md` setup and sandbox configuration to manage.

Q2 (Operational Characteristics). Three features define Codex's operational profile: cloud execution via git worktrees (parallel isolation with no local resource use), native context compaction (sessions of 24 hours or more, with roughly 30% fewer reasoning tokens and 47% lower input token use via Tool Search), and Fast Mode (up to 1.5× output speed on well-defined tasks). Rate limiting is not an issue on standard Plus plans (5/5), which is a meaningful advantage over competing tools. The main gaps are the Pro tier quota ceiling and the Security & IP score (1/5), which reflects the inherent concern with sending proprietary code to cloud-hosted infrastructure.

Q3 (Architectural Differentiators). Codex's git worktree architecture, cloud execution model, and native compaction offer capabilities that locally-executed tools do not have. Parallel task execution across multiple repositories, repeatable containerized environments, and the Tool Search token savings make it the most cost-effective option for high-volume automated pipelines. The official Codex plugin for Claude Code enables the dual-wielding workflow that practitioners consistently identify as the highest-productivity setup, sending design and architecture work to Claude Opus 4.6 and execution and verification work to Codex.

The overall finding is that Codex (GPT-5.4) is the strongest execution and debugging tool in the current agentic coding landscape. Its advantages in analytical depth, cloud-native parallelism, token efficiency at scale, and desktop automation are exactly where it complements Claude Code's strengths in architectural design and multi-agent orchestration. For teams willing to put in the work to set up `AGENTS.md` and sandbox configuration properly, Codex is a strong productivity tool whose analytical capabilities, running costs, and parallel execution model make it a practical part of any serious agentic development workflow.

References

[1] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021.

[2] C. E. Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" Proc. International Conference on Learning Representations (ICLR), 2024.

[3] D. Fryer et al., "Do Large Language Models Know What They Don't Know?" arXiv preprint arXiv:2402.10768, 2024.

[4] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot," arXiv preprint arXiv:2302.06590, 2023.

[5] McKinsey Global Institute, "The Economic Potential of Generative AI: The Next Productivity Frontier," McKinsey and Company, Jun. 2023.

[6] METR, "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity," METR Technical Report, 2025.

[7] J. W. Creswell and V. L. Plano Clark, *Designing and Conducting Mixed Methods Research*, 3rd ed. Thousand Oaks, CA: SAGE Publications, 2018.

[8] MorphLLM, "SWE-Bench Pro Leaderboard," 2026. [Online]. Available: https://www.morphllm.com/swe-bench-pro

[9] NxCode, "GPT-5.4 Complete Guide 2026: Features, Pricing, Benchmarks & How to Use," 2026. [Online]. Available: https://www.nxcode.io/resources/news/gpt-5-4-complete-guide-features-pricing-models-2026

[10] BenchLM, "GPT-5.4 Benchmark Results," 2026. [Online]. Available: https://benchlm.ai/models/gpt-5-4

[11] Vellum AI, "Claude Opus 4.6 Benchmarks and Model Card," 2026. [Online]. Available: https://www.vellum.ai/blog/claude-opus-4-6-benchmarks

[12] C. Nguyen, "Codex with GPT-5.4 vs Claude Code with Opus 4.6 – Why I Now Use Both," 2026. [Online]. Available: https://chandlernguyen.com/blog/2026/03/13/codex-gpt-5-4-vs-claude-code-opus-4-6-dual-wielding-ai-coding-tools/

[13] A. Holter, "GPT-5.4 Fast Mode Is the Best Part of the Release," 2026. [Online]. Available: https://adam.holter.com/gpt-5-4-fast-mode-is-the-best-part-of-the-release/

[14] T. Wiegold, "I Tested GPT 5.4 Against Every Rival – Here's My Honest Review," 2026. [Online]. Available: https://thomas-wiegold.com/blog/i-tested-gpt-5-4-against-every-rival/

[15] OpenAI Developer Community, "GPT-5.4 in Codex Feels Degraded," 2026. [Online]. Available: https://community.openai.com/t/gpt-5-4-in-codex-feels-degraded/1378747

[16] OpenAI Developer Community, "Understanding the New Codex Limit System After the April 9 Update," 2026. [Online]. Available: https://community.openai.com/t/understanding-the-new-codex-limit-system-after-the-april-9-update/1378768

[17] MindStudio, "OpenAI Codex Plugin for Claude Code: Cross-Provider Review," 2026. [Online]. Available: https://www.mindstudio.ai/blog/openai-codex-plugin-claude-code-cross-provider-review

[18] OpenAI, "Codex Developer Documentation," 2026. [Online]. Available: https://developers.openai.com/codex

[19] OpenAI, "Cloud Environments – Codex Web," 2026. [Online]. Available: https://developers.openai.com/codex/cloud/environments

[20] OpenAI, "Building More with GPT-5.1-Codex-Max," 2026. [Online]. Available: https://openai.com/index/gpt-5-1-codex-max/